# Συστήματα Υπολογισμού Υψηλών Επιδόσεων
# Project 5

**Τσόγκας Παναγιώτης**
ptsogkas@uth.gr
3672

**Νίκας Ιωάννης Ιάσων**
ioannikas@uth.gr
3771

**Τσιαντός Δημήτριος**
dtsiantos@uth.gr
3796

Ιανουάριος 2026

# Περιεχόμενα


**1 Ο αλγόριθμος** **3**

**2 Παραλληλοποίηση σε CPU με χρήση OpenMP** **4**

**3 Αρχική έκδοση σε GPU** **4**
3.1 Στρατηγική Παραλληλοποίησης . . . . . 4
3.2 Διαχωρισμός Kernels . . . . . 4
3.3 Global Memory και ειδικές συναρτήσεις . . . . . 4

**4 CUDA Streams** **5**

**5 Αλλαγή δομής δεδομένων σε Structure of Arrays** **6**

**6 Εφαρμογή tiling στην shared memory** **7**

**7 Read-Only Data Cache και Fused Multiply Add** **8**

**8 Coarsening** **9**

**9 Πολλαπλά GPUs** **9**

**10 Launch Bounds** **10**

**11 Εξάλειψη divergence** **10**

**12 Αποτελέσματα και μετρήσεις** **10**

# 1 Ο αλγόριθμος

Το πρόβλημα των N-σωμάτων αφορά την προσομοίωση της δυναμικής εξέλιξης ενός συστήματος $N$ σωματιδίων, τα οποία αλληλεπιδρούν μεταξύ τους μέσω φυσικών δυνάμεων. Στην περίπτωσή μας, εξετάζουμε τη βαρυτική αλληλεπίδραση σύμφωνα με τον Νόμο της Παγκόσμιας Έλξης του Νεύτωνα.
Η δύναμη $\vec{F}_{ij}$ που ασκείται σε ένα σώμα $i$ από ένα σώμα $j$ δίνεται από τον τύπο:

$$\vec{F}_{ij} = G\frac{m_i m_j}{|\vec{r}_{ij}|^3}\vec{r}_{ij} \tag{1}$$

όπου:

- $G$ είναι η σταθερά της βαρύτητας.
- $m_i, m_j$ είναι οι μάζες των σωμάτων.
- $\vec{r}_{ij} = \vec{r}_j - \vec{r}_i$ είναι το διάνυσμα θέσης από το $i$ στο $j$.

Η συνολική δύναμη $\vec{F}_i$ που δέχεται το σώμα $i$ είναι το άθροισμα των επιμέρους δυνάμεων από όλα τα υπόλοιπα $N-1$ σώματα:

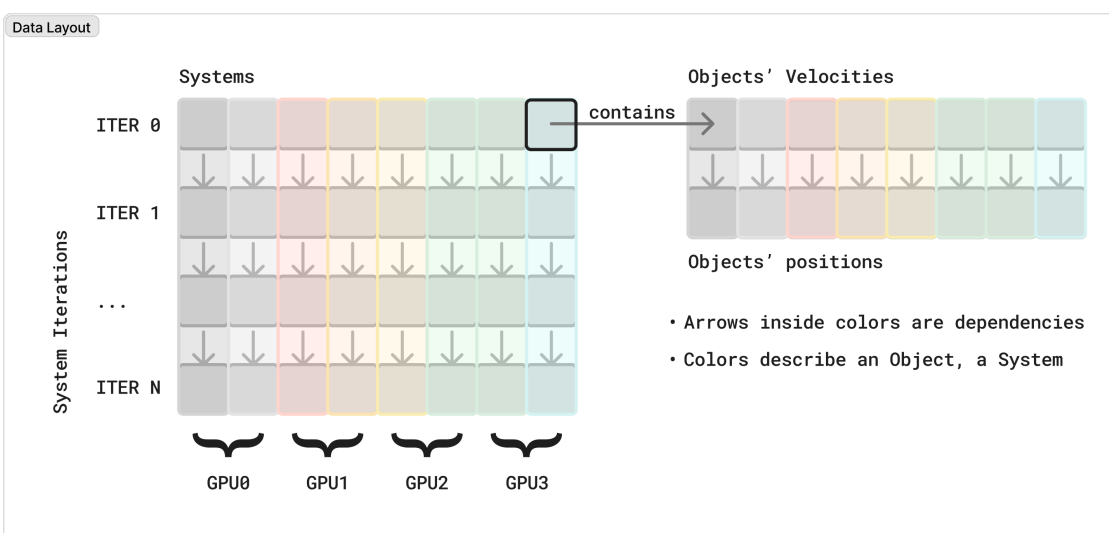
$$\vec{F}_i = \sum_{j=1, j\neq i}^{N} \vec{F}_{ij} \tag{2}$$

Η προσομοίωση της χρονικής εξέλιξης του συστήματος επιτυγχάνεται χωρίζοντας τον χρόνο σε μικρά, διακριτά βήματα διάρκειας $dt$. Σε κάθε χρονικό βήμα, ο αλγόριθμος εκτελείται σε δύο διακριτές φάσεις για το σύνολο των σωμάτων:

1. **Υπολογισμός Αλληλεπιδράσεων και Ταχυτήτων**
   Υπολογίζεται η συνολική δύναμη που ασκείται σε κάθε σώμα από όλα τα υπόλοιπα. Αμέσως μετά τον υπολογισμό της δύναμης (και άρα της επιτάχυνσης), ενημερώνεται η ταχύτητα του σώματος. Αυτό είναι το πιο απαιτητικό υπολογιστικά τμήμα της προσομοίωσης, καθώς απαιτεί ανάγνωση όλων των δεδομένων για κάθε σώμα.

2. **Ενημέρωση Θέσεων**
   Αφού έχουν υπολογιστεί οι νέες ταχύτητες για όλα τα σώματα, η δεύτερη φάση ενημερώνει τις συντεταγμένες κάθε σώματος βάσει της τρέχουσας ταχύτητάς του.

Η ροή αυτή επαναλαμβάνεται για τον επιθυμητό αριθμό βημάτων της προσομοίωσης όπως φαίνεται στο παρακάτω διάγραμμα:



# 2 Παραλληλοποίηση σε CPU με χρήση OpenMP

Ο παραλληλισμός εφαρμόστηκε στο εξωτερικό επίπεδο της προσομοίωσης, δηλαδή στο βρόχο που διατρέχει τα ανεξάρτητα συστήματα, και όχι στον εσωτερικό βρόχο των σωμάτων. Με αυτή την επιλογή καταφέρνουμε να έχουμε μικρό overhead διότι δεν απαιτείται η συχνή δημιουργία και καταστροφή νημάτων. Επιπλέον αποφεύγουμε την ανάγκη για συγχρονισμό, που θα μείωνε την απόδοση, καθώς τα συστήματα μεταξύ τους είναι πλήρως ανεξάρτητα και δεν υπάρχουν race conditions. Εφαρμόζουμε dynamic scheduling ώστε να εξισορροπήσουμε τους υπολογισμούς στα νήματα. Έπειτα με την χρήση του `pragma omp simd`, που χρησιμοποιεί SIMD εντολές για να εκτελέσει πράξεις σε μεγαλύτερους registers, εφαρμόσαμε παραλληλισμό σε επίπεδο εντολών.

Τέλος, ο αρχικός κώδικας χρησιμοποιούσε τη δομή **Array of Structures**, όπου τα δεδομένα κάθε σώματος ήταν ομαδοποιημένα σε ένα struct. Αυτή η προσέγγιση δημιουργεί μεγάλα άλματα στη μνήμη κατά την ανάγνωση συγκεκριμένων πεδίων (π.χ. μόνο των συντεταγμένων $x$). Για να βελτιστοποιήσουμε την απόδοση μετατρέψαμε την δομή σε **Structure of Arrays (SoA)**. Έτσι όλα τα $x$ αποθηκεύονται συνεχόμενα στην μνήμη. Έτσι πετυχαίνουμε καλύτερη τοπικότητα καθώς η cache γεμίζει με δεδομένα που είναι πιο πιθανό να χρησιμοποιήσουμε. Η συγκεκριμένη βελτιστοποίηση εφαρμόζεται και παρακάτω στις εκδόσεις για GPU για παρόμοιους λόγους.

Πίνακας 1: Μέση τιμή 25 εκτελέσεων και αφαίρεση ελάχιστης και μέγιστης μέτρησης

| **Έκδοση** | **Throughput** (Billion Interactions/s) | **Τυπική Απόκλιση** ($\sigma$) | **Execution time** (s) | **Improvement** |
|---|---|---|---|---|
| Sequential CPU | 0.1482 | 0.13 | 293.255 | x1 |
| OpenMP CPU | 10.6469 | 0.0689 | 4.039 | x71.84 |

# 3 Αρχική έκδοση σε GPU

Στην πρώτη μας υλοποίηση για την GPU, ο στόχος ήταν η μεταφορά του αλγορίθμου από τη CPU στο περιβάλλον CUDA με τον πιο άμεσο τρόπο. Η στρατηγική βασίζεται στην αντιστοίχιση ένος νήματος ανά σώμα.

## 3.1 Στρατηγική Παραλληλοποίησης

Σε αντίθεση με την OpenMP υλοποίηση όπου παραλληλοποιήσαμε τα συστήματα, εδώ εκμεταλλευόμαστε τον μαζικό παραλληλισμό της GPU για να υπολογίσουμε τις αλληλεπιδράσεις εντός του κάθε συστήματος. Δημιουργούμε ένα grid νημάτων όπου κάθε νήμα αναλαμβάνει αποκλειστικά ένα σώμα ενός γαλαξία. Κάθε νήμα διατρέχει όλα τα $N$ σώματα του συστήματος, διαβάζει τις θέσεις τους και αθροίζει τις δυνάμεις. Δεδομένου ότι κάθε νήμα γράφει σε διαφορετική θέση μνήμης, δεν απαιτούνται μηχανισμοί συγχρονισμού κατά την εγγραφή των αποτελεσμάτων.

## 3.2 Διαχωρισμός Kernels

Η προσομοίωση χωρίζεται σε δύο ξεχωριστούς πυρήνες, `bodyForceKernel` και `integrateKernel`. Αυτός ο διαχωρισμός είναι απαραίτητος διότι λειτουργεί ως ένα φράγμα συγχρονισμού. Πρέπει να βεβαιωθούμε ότι έχουν υπολογιστεί οι ταχύτητες για όλα τα σώματα πριν προχωρήσουμε στην ενημέρωση των θέσεών τους για το επόμενο βήμα.

## 3.3 Global Memory και ειδικές συναρτήσεις

Σε αυτή την έκδοση, όλα τα δεδομένα (θέσεις και ταχύτητες) βρίσκονται στην global memory της GPU. Κάθε νήμα χρειάζεται να διαβάσει τη θέση και των $N$ σωμάτων από την Global Memory για να υπολογίσει τη δύναμη στο δικό του σώμα. Αυτό οδηγεί σε $N$ αναγνώσεις για κάθε ένα από τα $N$ νήματα, συνολικά δηλαδή $N^2$ προσπελάσεις στην Global Memory, η οποία έχει μεγάλη καθυστέρηση και έτσι οδηγούμαστε σε μειωμένη απόδοση. Αυτό αποτελεί και το κύριο bottleneck της συγκεκριμένης υλοποίησης.

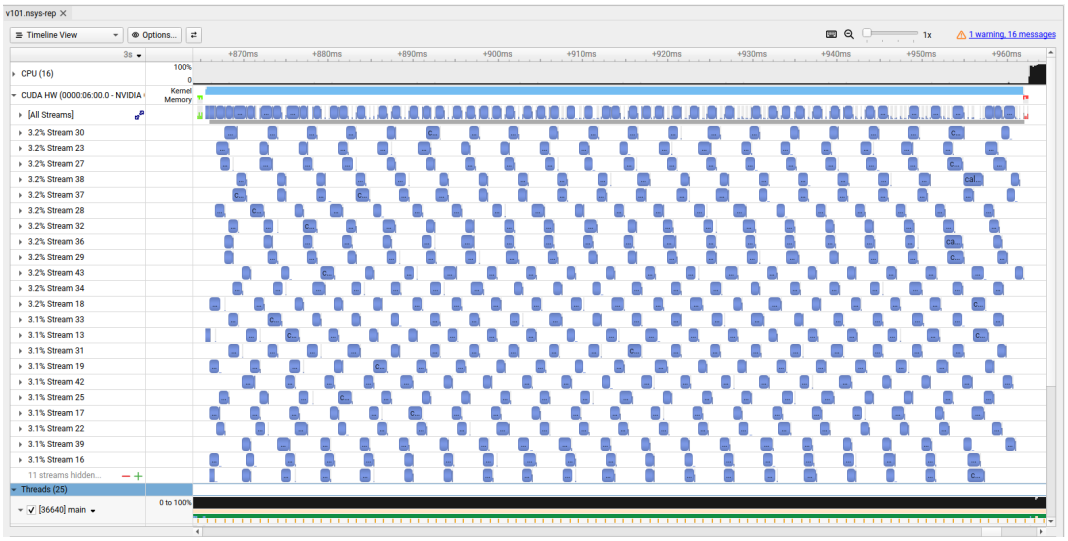
Για τον υπολογισμό του αντίστροφου της τετραγωνικής ρίζας, αντικαταστήσαμε την τυπική διαίρεση `1.0f / sqrtf(...)` με την ενσωματωμένη συνάρτηση (intrinsic) `rsqrtf()`. Η συνάρτηση αυτή εκτελείται απευθείας από τις μονάδες ειδικών λειτουργιών (SFU) της GPU και είναι σημαντικά ταχύτερη, με ελάχιστη απώλεια ακρίβειας.

Πίνακας 2: Μέση τιμή 25 εκτελέσεων και αφαίρεση ελάχιστης και μέγιστης μέτρησης

| **Έκδοση** | **Throughput** (Billion Interactions/s) | **Τυπική Απόκλιση** ($\sigma$) | **Execution time** (s) | **Improvement** |
|---|---|---|---|---|
| Sequential CPU | 0.1482 | 0.13 | 293.255 | x1 |
| OpenMP CPU | 10.6469 | 0.0689 | 4.039 | x71.84 |
| Naive GPU | 29.2351 | 1.3894 | 1.473 | x197.26 |

# 4 CUDA Streams

Για την πλήρη αξιοποίηση των πόρων της GPU, χρησιμοποιούμε streams ώστε να εκτελούμε παράλληλα στην συσκευή πολλαπλά kernels. Εφόσον τα σώματα μεταξύ διαφορετικών συστημάτων δεν έχουν εξαρτήσεις, η χρήση streams επιτρέπει στον scheduler της GPU να εκτελεί τα kernels παράλληλα.



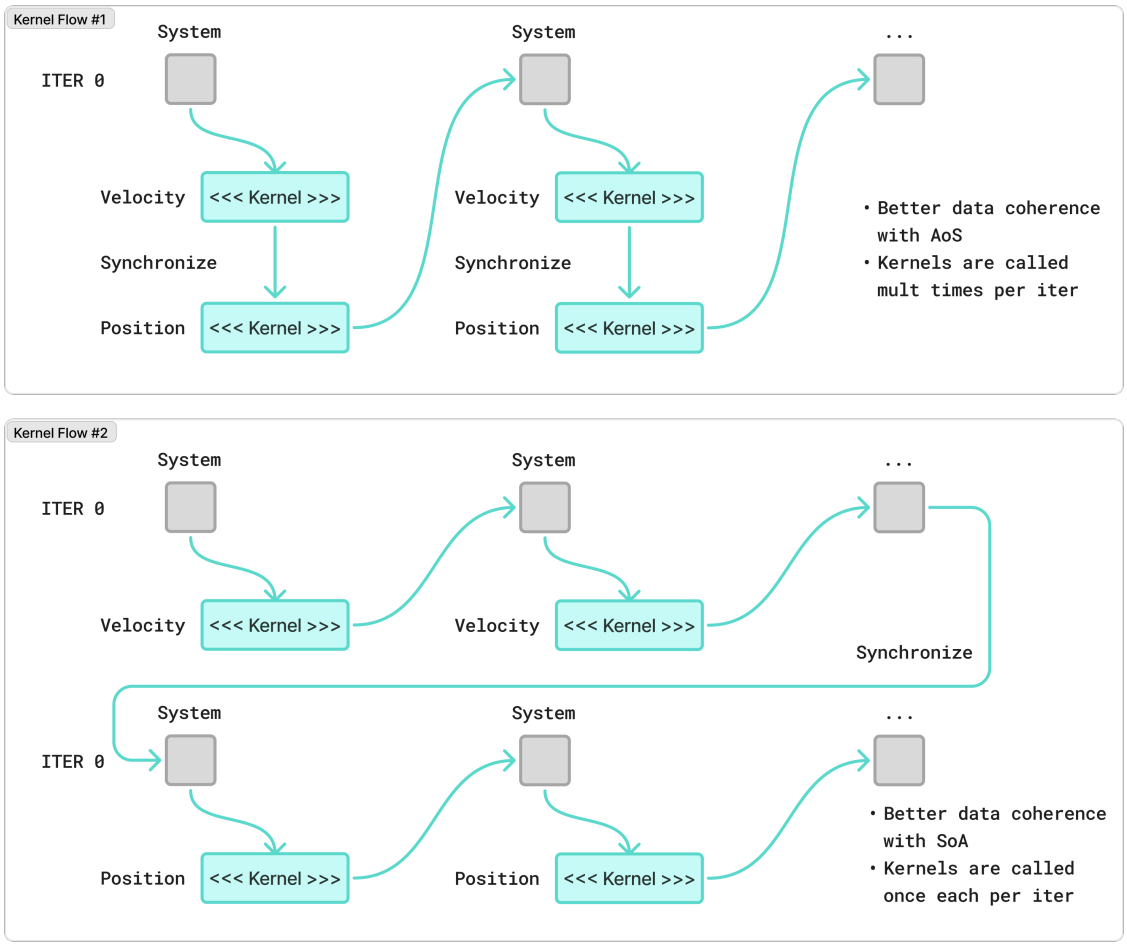

Σχήμα 1: Gantt chart της εκτέλεσης των kernels. Κάθε μπλε τετράγωνο αντιπροσωπεύει μία εκτέλεση του `bodyForceKernel` kernel.

Πίνακας 3: Μέση τιμή 25 εκτελέσεων και αφαίρεση ελάχιστης και μέγιστης μέτρησης

| **Έκδοση** | **Throughput** (Billion Interactions/s) | **Τυπική Απόκλιση** ($\sigma$) | **Execution time** (s) | **Improvement** |
|---|---|---|---|---|
| Sequential CPU | 0.1482 | 0.13 | 293.255 | x1 |
| OpenMP CPU | 10.6469 | 0.0689 | 4.039 | x71.84 |
| Naive GPU | 29.2351 | 1.3894 | 1.473 | x197.26 |
| Streams | 40.0133 | 0.4182 | 1.0735 | x269.99 |

# 5 Αλλαγή δομής δεδομένων σε Structure of Arrays

Για τη δομή των kernels καταλήξαμε σε δύο εκδοχές, εκ των οποίων επιλέξαμε το Kernel Flow #2, καθώς το Kernel Flow #1 απαιτεί συχνότερους συγχρονισμούς. Η δομή αυτή αποδεικνύεται αποδοτικότερη σε συνδυασμό με οργάνωση των δεδομένων σε μορφή Structure of Arrays (SoA), και για τον λόγο αυτό υιοθετήθηκε η συγκεκριμένη αναπαράσταση δεδομένων.




Όπως και στην έκδοση OpenMP, αλλάξαμε τη δομή Array of Structures σε Structure of Arrays. Σε περιβάλλον CUDA, η αλλαγή αυτή είναι ακόμα πιο σημαντική λόγω του τρόπου που λειτουργεί η global memory της GPU.

Όταν ένα warp διαβάζει δεδομένα από την Global Memory, η GPU προσπαθεί να εξυπηρετήσει όλες τις αιτήσεις με ένα μοναδικό memory transaction, αρκεί οι διευθύνσεις να είναι συνεχόμενες. Οπότε στην περίπτωση AoS, τα δεδομένα θέσης $x$ για το νήμα $k$ και το νήμα $k+1$ απέχουν μεταξύ τους όσο το μέγεθος ολόκληρου του struct Body. Αυτό οδηγεί σε μη αποδοτική χρήση του bandwidth (strided access). Ενώ στην περίπτωση SoA τα δεδομένα $x$ είναι αποθηκευμένα σε συνεχόμενες θέσεις μνήμης. Όταν το νήμα $k$ διαβάζει το `soa.x[k]` και το νήμα $k+1$ διαβάζει το `soa.x[k+1]`, οι διευθύνσεις είναι γειτονικές. Αυτό επιτρέπει **Coalesced Accesses**, μειώνοντας κατά πολύ τον αριθμό των memory transactions.

Πρέπει να σημειωθεί ότι η χρήση SoA εισάγει overhead στον host για την αρχική και τελική μετατροπή των δεδομένων. Ωστόσο, το κέρδος από την ταχύτερη προσπέλαση μνήμης στην GPU υπερκαλύπτει αυτό το κόστος, ειδικά για μεγάλο αριθμό επαναλήψεων.

Πίνακας 4: Μέση τιμή 25 εκτελέσεων και αφαίρεση ελάχιστης και μέγιστης μέτρησης

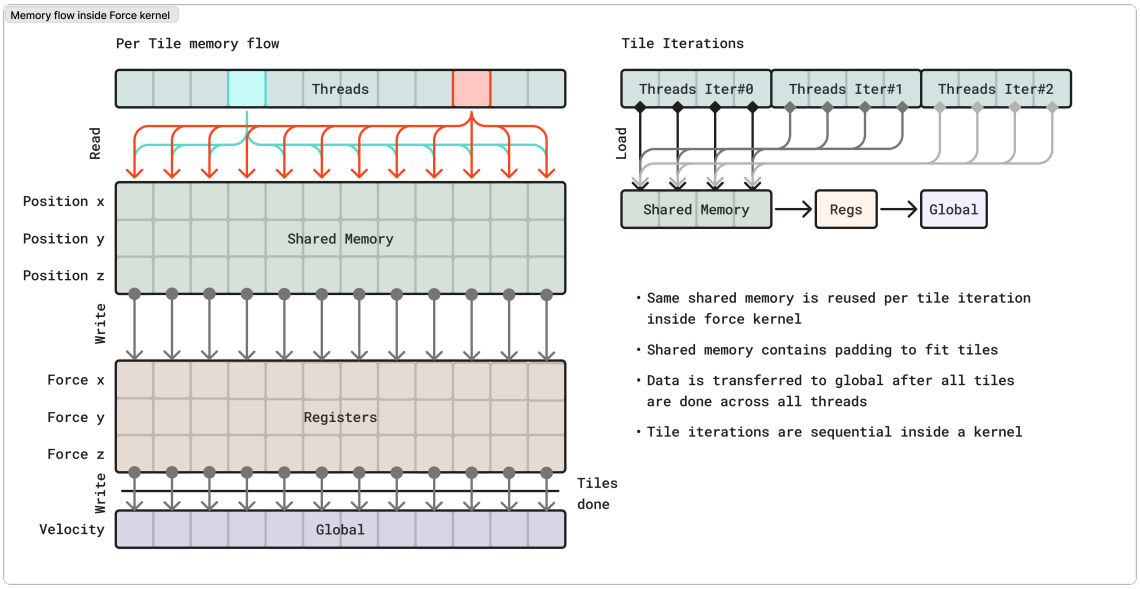
| **Έκδοση** | **Throughput** (Billion Interactions/s) | **Τυπική Απόκλιση** ($\sigma$) | **Execution time** (s) | **Improvement** |
|---|---|---|---|---|
| Sequential CPU | 0.1482 | 0.13 | 293.255 | x1 |
| OpenMP CPU | 10.6469 | 0.0689 | 4.039 | x71.84 |
| Naive GPU | 29.2351 | 1.3894 | 1.473 | x197.26 |
| Streams | 40.0133 | 0.4182 | 1.0735 | x269.99 |
| Structure of Arrays | 40.9047 | 0.4630 | 1.0502 | x276.01 |

# 6 Εφαρμογή tiling στην shared memory

Με τον τρόπο που προσπελαύνονται τα δεδομένα, ιδανικά θα θέλαμε να τοποθετήσουμε όλα τα δεδομένα ενός συστήματος στο shared memory, ώστε να εργαζόμαστε αποκλειστικά πάνω σε ένα ενιαίο αντίγραφο που βρίσκεται εκεί. Στην πράξη, όμως, κάθε kernel δεν διαθέτει επαρκή shared memory για να υποστηρίξει μια τέτοια προσέγγιση. Για τον λόγο αυτό, το πρόβλημα διασπάται σε tiles, επιτρέποντας την επαναχρησιμοποίηση ενός shared memory block σε κάθε εκτέλεση.

Στην Εικόνα 2 παρουσιάζεται ο τρόπος διαχείρισης των δεδομένων μέσα στο πλαίσιο ενός kernel. Συγκεκριμένα, διακρίνονται δύο διακριτές φάσεις. Στην πρώτη φάση, κάθε thread φορτώνει τα δεδομένα ενός σώματος στο shared memory. Στη δεύτερη φάση, τα threads χρησιμοποιούν τις πληροφορίες που βρίσκονται στο shared memory για να υπολογίσουν μερικά αποτελέσματα για τις μεταβλητές $F_x$, $F_y$ και $F_z$.

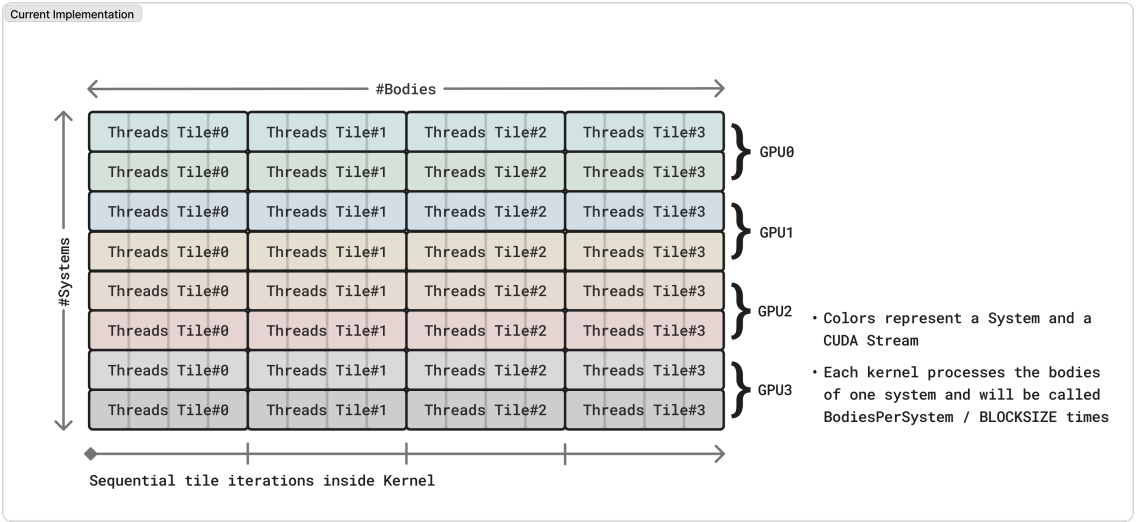
Τα μερικά αυτά αποτελέσματα συσσωρεύονται καθώς επεξεργάζονται διαδοχικά όλα τα tiles, και το τελικό αποτέλεσμα γράφεται πίσω στο global memory μόνο αφού ολοκληρωθεί ο υπολογισμός για όλα τα tiles.




Σχήμα 2: Shared memory flow

Με τη μέθοδο αυτή καταλήγουμε σε μια υλοποίηση στην οποία κάθε κλήση kernel υπολογίζει τις ταχύτητες των σωμάτων ενός συστήματος σε blocks μεγέθους `BLOCK_SIZE`. Η προσπέλαση των δεδομένων πραγματοποιείται σε πολλαπλά grids, επιτρέποντας την τμηματική επεξεργασία του προβλήματος.

Τα kernels αυτά χρησιμοποιούν streams, ώστε να μπορούν να εκτελούνται παράλληλα. Ο τρόπος προσπέλασης των δεδομένων απεικονίζεται στην Εικόνα 3.




Σχήμα 3: Data access pattern

Πίνακας 5: Μέση τιμή 25 εκτελέσεων και αφαίρεση ελάχιστης και μέγιστης μέτρησης

| **Έκδοση** | **Throughput** (Billion Interactions/s) | **Τυπική Απόκλιση** ($\sigma$) | **Execution time** (s) | **Improvement** |
|---|---|---|---|---|
| Sequential CPU | 0.1482 | 0.13 | 293.255 | x1 |
| OpenMP CPU | 10.6469 | 0.0689 | 4.039 | x71.84 |
| Naive GPU | 29.2351 | 1.3894 | 1.473 | x197.26 |
| Streams | 40.0133 | 0.4182 | 1.0735 | x269.99 |
| Structure of Arrays | 40.9047 | 0.4630 | 1.0502 | x276.01 |
| Tiling | 54.6848 | 0.9807 | 0.7856 | x368.99 |

# 7 Read-Only Data Cache και Fused Multiply Add

Για την ανάγνωση των θέσεων των σωμάτων από την μνήμη, αξιοποιούμε την Read-Only Data Cache της συσκευής με την χρήση της συνάρτησης `__ldg()`. Εφόσον οι θέσεις των σωμάτων μόνο διαβάζονται από την μνήμη, η αποθήκευσή τους στην read-only cache χρησιμοποιεί χωριστό bus για την μεταφορά των δεδομένων στους SMs, οπότε εξοικονομούμε χώρο και bandwidth από την L1 cache.

Επιπλέον βελτίωση στην επίδοση προσφέρει και η εντολή fused multiply add με την χρήση της συνάρτησης `fmaf()`. Αυτή η συνάρτηση εκτελεί την πράξη του πολλαπλασιασμού και της πρόσθεσης σε ένα instruction και χρησιμοποιείται τόσο στον kernel του υπολογισμού των δυνάμεων, όσο και στον υπολογισμό των θέσεων.

Πίνακας 6: Μέση τιμή 25 εκτελέσεων και αφαίρεση ελάχιστης και μέγιστης μέτρησης

| **Έκδοση** | **Throughput** (Billion Interactions/s) | **Τυπική Απόκλιση** ($\sigma$) | **Execution time** (s) | **Improvement** |
|---|---|---|---|---|
| Sequential CPU | 0.1482 | 0.13 | 293.255 | x1 |
| OpenMP CPU | 10.6469 | 0.0689 | 4.039 | x71.84 |
| Naive GPU | 29.2351 | 1.3894 | 1.473 | x197.26 |
| Streams | 40.0133 | 0.4182 | 1.0735 | x269.99 |
| Structure of Arrays | 40.9047 | 0.4630 | 1.0502 | x276.01 |
| Tiling | 54.6848 | 0.9807 | 0.7856 | x368.99 |
| Special Functions | 84.6815 | 1.9990 | 0.5073 | x571.40 |

# 8 Coarsening

Αναθέτοντας σε κάθε thread περισσότερη δουλειά, βελτιώνεται αισθητά η επίδοση. Αυτό οφείλεται στην καλύτερη αξιοποίηση των δεδομένων εντός του kernel και στο instruction level parallelism (ILP). Αντί ένα thread να υπολογίζει την δύναμη ενός σώματος, θα μπορούσε να υπολογίζει από δύο ή παραπάνω σώματα σε ένα kernel. Αυτό προσφέρει καλύτερα memory accesses καθώς κάθε σύνολο στοιχείων που φορτώνονται από την shared memory επαναχρησιμοποιούνται για πολλαπλούς υπολογισμούς δυνάμεων ταυτόχρονα, μειώνοντας τα memory accesses. Είναι σημαντικό τα memory accesses να είναι coallesced, παίρνοντας το i-οστό στοιχείο από κάθε block. Ο scheduler μπορεί, πλέον, να κάνει pipeline τις εντολές μέσα σε ένα kernel. Το μειωνέκτημα είναι πως αυξάνεται η χρήση των registers σε κάθε kernel call, αλλά τα παραπάνω πλεονεκτήματα υπερτερούν, με αποτέλεσμα να μειώνεται ο χρόνος εκτέλεσης. Ύστερα από δοκιμές, παρατηρήσαμε πως το καλύτερο coarsening factor είναι δύο bodies ανά thread.

Πίνακας 7: Μέση τιμή 25 εκτελέσεων και αφαίρεση ελάχιστης και μέγιστης μέτρησης

| **Έκδοση** | **Throughput** (Billion Interactions/s) | **Τυπική Απόκλιση** ($\sigma$) | **Execution time** (s) | **Improvement** |
|---|---|---|---|---|
| Sequential CPU | 0.1482 | 0.13 | 293.255 | x1 |
| OpenMP CPU | 10.6469 | 0.0689 | 4.039 | x71.84 |
| Naive GPU | 29.2351 | 1.3894 | 1.473 | x197.26 |
| Streams | 40.0133 | 0.4182 | 1.0735 | x269.99 |
| Structure of Arrays | 40.9047 | 0.4630 | 1.0502 | x276.01 |
| Tiling | 54.6848 | 0.9807 | 0.7856 | x368.99 |
| Special Functions | 84.6815 | 1.9990 | 0.5073 | x571.40 |
| Coarsening | 72.1287 | 1.6262 | 0.5957 | x486.69 |

# 9 Πολλαπλά GPUs

Για τη χρήση πολλαπλών GPU δοκιμάσαμε υλοποιήσεις τόσο με CUDA streams όσο και με OpenMP threads, και καταλήξαμε στη χρήση του OpenMP λόγω απλότητας. Συγκεκριμένα, ορίζουμε ένα parallel region με αριθμό threads ίσο με τον αριθμό των διαθέσιμων GPU, μέχρι το ανώτατο όριο των `GPU_MAX`.

Η λογική μεταξύ των threads είναι ίδια και το συνολικό έργο διαμοιράζεται σε ίσα τμήματα, όσα και οι GPU που δεσμεύονται. Αυτό είναι εφικτό επειδή ο υπολογισμός του προβλήματος n-body μεταξύ ανεξάρτητων συστημάτων είναι embarrassingly parallel. Ως αποτέλεσμα, επιτρέπεται ο διαχωρισμός του προβλήματος σε ισομεγέθη τμήματα, τα οποία διαχειρίζονται διαφορετικά CPU threads, το καθένα συνδεδεμένο με ξεχωριστή GPU.

Αξίζει να σημειωθεί ότι σε σύστημα με CUDA 13.0 ο μεταγλωττιστής NVCC υποστηρίζει την εντολή `cudaInitDevice()`, η οποία έχει αντικατασταθεί από την `cudaSetDevice()`, ακολουθούμενη από μια κλήση `cudaFree(0)`. Σύμφωνα με σχετικές αναφορές σε forums, η κλήση αυτή εξαναγκάζει τη δημιουργία ενός CUDA context, με τρόπο αντίστοιχο της εντολής `cuDevicePrimaryCtxRetain()`.

Πίνακας 8: Μέση τιμή 25 εκτελέσεων και αφαίρεση ελάχιστης και μέγιστης μέτρησης

| **Έκδοση** | **Throughput** (Billion Interactions/s) | **Τυπική Απόκλιση** ($\sigma$) | **Execution time** (s) | **Improvement** |
|---|---|---|---|---|
| Sequential CPU | 0.1482 | 0.13 | 293.255 | x1 |
| OpenMP CPU | 10.6469 | 0.0689 | 4.039 | x71.84 |
| Naive GPU | 29.2351 | 1.3894 | 1.473 | x197.26 |
| Streams | 40.0133 | 0.4182 | 1.0735 | x269.99 |
| Structure of Arrays | 40.9047 | 0.4630 | 1.0502 | x276.01 |
| Tiling | 54.6848 | 0.9807 | 0.7856 | x368.99 |
| Special Functions | 84.6815 | 1.9990 | 0.5073 | x571.40 |
| Coarsening | 72.1287 | 1.6262 | 0.5957 | x486.69 |
| Multiple GPUs | 322.2653 | 19.5558 | 0.1338 | x2174.52 |

# 10 Launch Bounds

Για τη μεγιστοποίηση της απόδοσης του kernel χρησιμοποιήθηκε η οδηγία `__launch_bounds__()`. Κατά τη διαδικασία μεταγλώττισης (με χρήση του flag `-Xptxas -v`) παρατηρήθηκε ότι η συγκεκριμένη ρύθμιση οδηγεί σε αυξημένη χρήση καταχωρητών (registers) ανά νήμα, σε σύγκριση με ρυθμίσεις που στοχεύουν σε μέγιστο occupancy, όπως η επιλογή ελάχιστου αριθμού blocks ανά SM ίσου με 8.

Παρότι η μείωση του ελάχιστου αριθμού blocks ανά Streaming Multiprocessor μειώνει το θεωρητικό occupancy, στην πράξη επιτρέπει στον compiler να αποφύγει το register spilling. Η αυξημένη διαθεσιμότητα registers επιτρέπει στα νήματα να εκτελούνται ταχύτερα, αντισταθμίζοντας την απώλεια παραλληλισμού σε επίπεδο block και οδηγώντας τελικά σε βελτιωμένο συνολικό χρόνο εκτέλεσης.

Παρακάτω παρουσιάζεται παράδειγμα υλοποίησης με coarsening ίσο με 4 και διαφορετικές οδηγίες `launch_bounds`, το οποίο είχε ως αποτέλεσμα τη μείωση της απόδοσης περίπου στο μισό. Η διαφορά στην απόδοση εξηγείται άμεσα από τις πληροφορίες του `ptxas`, όπου φαίνεται καθαρά η εμφάνιση εκτεταμένου register spilling στην πρώτη περίπτωση.

Οδηγία `__launch_bounds__(THREADS_PER_BLOCK, 4)`

```
    136 bytes stack frame, 188 bytes spill stores, 180 bytes spill loads
ptxas info    : Used 64 registers, used 1 barriers, 136 bytes cumulative stack size,
                3072 bytes smem, 412 bytes cmem[0]
ptxas info    : Compile time = 24.877 ms
```

Οδηγία `__launch_bounds__(THREADS_PER_BLOCK, 2)`

```
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info    : Used 128 registers, used 1 barriers,
                3072 bytes smem, 412 bytes cmem[0]
ptxas info    : Compile time = 25.730 ms
```

# 11 Εξάλειψη divergence

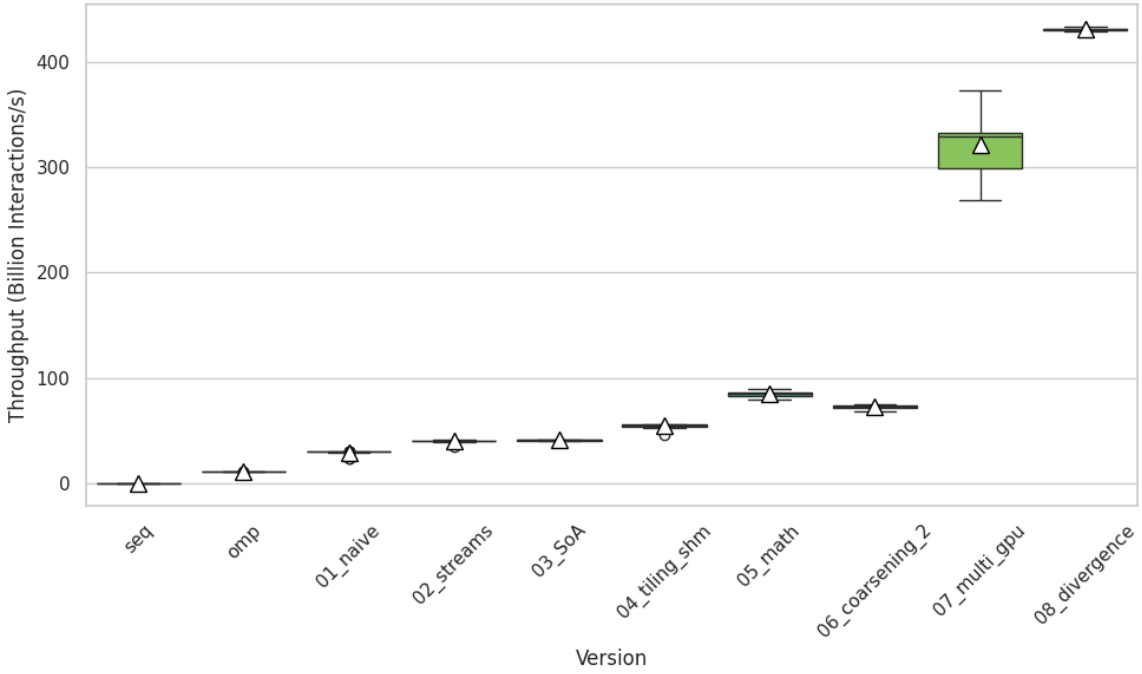
Στους kernels χρησιμοποιούμε διάφορες συνθήκες που ελέγχουν πως οι δείκτες είναι εντός ορίων. Αυτό, ωστόσο, δημιουργεί το φαινόμενο του divergence, κάτι που επιλύουμε με την προσθήκη padding. Η τεχνική του padding εφαρμόζεται για να ευθυγραμμιστεί ο αριθμός των σωμάτων με το γινόμενο του μεγέθους του block και του συντελεστή coarsening. Έτσι, όλα τα threads εντός των warps εκτελούν πάντα τις ίδιες εντολές, χωρίς να ελέγχουν αν είναι εντός ορίων.

# 12 Αποτελέσματα και μετρήσεις

Για να μετρήσουμε την απόδοση κάθε βελτιστοποίησης που πραγματοποιήσαμε, τρέξαμε κάθε έκδοση 25 φορές κρατώντας τον μέσο όρο του throughput αφαιρώντας τον μεγαλύτερο και μικρότερο outlier. Παρακάτω φαίνονται τα αποτελέσματα:

Πίνακας 9: Μέση τιμή 25 εκτελέσεων και αφαίρεση ελάχιστης και μέγιστης μέτρησης

| **Έκδοση** | **Throughput** (Billion Interactions/s) | **Τυπική Απόκλιση** ($\sigma$) | **Execution time** (s) | **Improvement** |
|---|---|---|---|---|
| Sequential CPU | 0.1482 | 0.13 | 293.255 | x1 |
| OpenMP CPU | 10.6469 | 0.0689 | 4.039 | x71.84 |
| Naive GPU | 29.2351 | 1.3894 | 1.473 | x197.26 |
| Streams | 40.0133 | 0.4182 | 1.0735 | x269.99 |
| Structure of Arrays | 40.9047 | 0.4630 | 1.0502 | x276.01 |
| Tiling | 54.6848 | 0.9807 | 0.7856 | x368.99 |
| Special Functions | 84.6815 | 1.9990 | 0.5073 | x571.40 |
| Coarsening | 72.1287 | 1.6262 | 0.5957 | x486.69 |
| Multiple GPUs | 322.2653 | 19.5558 | 0.1338 | x2174.52 |
| Divergence | 431.2731 | 1.0297 | 0.0997 | x2910.07 |




Σχήμα 4: Σύγκριση των διαφορετικών εκδόσεων